INTERNAL FOUNDER REVIEW

Business/Product Report

Repository-derived assessment of the product as it exists today.

Product	Outbound Copilot
Repository	D:/Project/CEG
Date	2026-04-09

1. Executive Summary

Outbound Copilot is a workspace-based web application for AI-assisted outbound sales execution. Based on the repository, it is designed to help SDR teams, SaaS founders, and lead generation agencies research prospects, generate outbound email sequences, ingest and classify replies, draft follow-up responses, and manage those workflows inside a shared workspace. Evidence appears across the public marketing pages, the protected app flows, the shared domain packages, and the database schema in `README.md`, `apps/web/app/page.tsx`, `apps/web/app/app/campaigns/[campaignId]/prospects/[prospectId]/page.tsx`, `packages/database/migrations/0001_initial_schema.sql`, and `packages/validation/src/entities.ts`.

The product is no longer a placeholder prototype. It has a meaningful end-to-end application surface: authentication, onboarding, sender profiles, campaigns, prospects, prospect research, sequence generation, reply analysis, reply drafting, billing, Gmail connection/import, internal admin tooling, demo data, and data export foundations. At the same time, the repo also shows clear limits: there is no full email sending workflow yet, queueing is still “async-ready” rather than extracted to background infrastructure, Microsoft 365 is scaffolded but not implemented, and some repository paths still rely on in-memory adapters rather than a fully migrated Postgres-backed stack (`ARCHITECTURE.md`, `apps/web/lib/server/inbox/service.ts`, `packages/jobs/src/index.ts`).

Overall maturity is best described as an advanced MVP approaching early production readiness: the app has real product breadth and internal discipline, but it still carries several “foundation complete, scale-out later” choices that should be understood honestly.

2. What the Business Is Today

The implemented product is a SaaS-style outbound copilot for business users who run email-based outbound motions. This is confirmed by the app shell, feature boundaries, database model, and pricing/billing implementation in `apps/web/app/page.tsx`, `apps/web/app/pricing/page.tsx`, `apps/web/lib/server/billing.ts`, and `packages/billing/src/plans.ts`.

Confirmed from the repo:

- Product type: a multi-workspace web application built with Next.js and backed by Supabase/Postgres-style relational storage (`apps/web/package.json`, `packages/database/migrations/0001_initial_schema.sql`).
- Core use case: help a user set up sender context, create campaigns and prospects, run prospect research, generate outbound sequences, capture replies, analyze intent/objections, and draft response options (`apps/web/app/app/onboarding/page.tsx`, `apps/web/lib/server/prospect-research.ts`, `apps/web/lib/server/sequences.ts`, `apps/web/lib/server/replies.ts`).
- Target users explicitly named in the code and copy: SDRs, SaaS founders, lead generation agencies, and a basic fallback mode (`README.md`, `apps/web/app/page.tsx`, `infrastructure/demo-data/fixtures.ts`).

Inferred from code/configuration:

- Likely business model: subscription SaaS with tiered plans (`free`, `pro`, `agency`) and outcome-oriented feature/usage gating, rather than token-credit pricing (`packages/billing/src/plans.ts`, `apps/web/app/pricing/page.tsx`).
- Likely sales motion: founder-led or small-team commercial software sold initially to individual teams or agencies rather than large enterprise procurement, because the app includes team roles and institutional controls but keeps the operational surface intentionally lightweight (`apps/web/app/app/settings/page.tsx`, `ARCHITECTURE.md`).

3. Problem It Solves

Based on the actual workflows in the repo, the product appears to solve four related problems:

1. Manual outbound research and message creation are slow and inconsistent.

The app fetches and structures prospect website information, turns it into a company profile, and feeds that into sequence generation (`apps/web/lib/server/prospect-research.ts` , `packages/research-engine/src/contracts.ts` , `apps/web/lib/server/openai-research-provider.ts`).

1. Generic AI copy is risky for serious outbound.

   The system repeatedly emphasizes sender-aware personalization, evidence-backed claims, schema validation, quality checks, and guardrails against unsupported claims or pushiness (`README.md` , `apps/web/lib/server/openai-sequence-provider.ts` , `apps/web/lib/server/openai-reply-provider.ts` , `packages/validation/src/entities.ts`).

1. Teams need a unified place to manage outbound threads and reply handling.

   The prospect detail screen combines research, sequences, messages, reply analysis, drafts, Gmail import, and draft-to-inbox steps into one workflow (`apps/web/app/app/campaigns/[campaignId]/prospects/[prospectId]/page.tsx` , `apps/web/lib/server/inbox/service.ts`).

1. The business wants to learn from performance over time.

   The repo stores usage events, audit events, training signals, campaign performance snapshots, evaluation fixtures, and dataset export foundations (`packages/database/migrations/0001_initial_schema.sql` , `DATA_STRATEGY.md` , `packages/testing/tests/run-evals.ts`).

---

4. Current Feature Set

Area	Feature	What it does	Status	Evidence
Authentication	Supabase magic-link sign-in	Supports sign-in, callback handling, protected routes, and server-side session handling	Implemented	`apps/web/app/sign-in/page.tsx` , `apps/web/lib/s` `DEPLOYMENT.md`
Workspace management	Workspace-scoped app with roles	Supports workspaces and roles (`owner` , `admin` , `member`) with membership records	Implemented	`packages/database/migrations/0001_initial_sch` `packages/validation/src/entities.ts` , `apps/web`
Onboarding	First-run onboarding flow	Guides users through workspace confirmation, user type, sender profile/basic mode, first campaign, and first prospect	Implemented	`apps/web/app/app/onboarding/page.tsx`
Sender profiles	Sender-aware personalization setup	Stores sender profile type, company context, proof points, goals, and tone preferences	Implemented	`apps/web/app/app/sender-profiles/page.tsx` , `packages/validation/src/entities.ts`
Campaigns	Campaign creation and management	Stores objectives, ICP/persona, offer summary, tone/framework preferences, and status	Implemented	`apps/web/app/app/campaigns/page.tsx` , `packages/database/migrations/0001_initial_sch`
Prospects	Prospect record management	Stores people/companies, status, source, company site, and campaign linkage	Implemented	`apps/web/app/app/campaigns/[campaignId]/page.` `packages/database/migrations/0001_initial_sch`
Research	Prospect website research and company-profile extraction	Fetches public websites, extracts/summarizes company profile data, stores evidence/confidence/flags	Implemented	`apps/web/lib/server/prospect-research.ts` , `pac` `engine/src/contracts.ts` , `apps/web/lib/server/` `provider.ts`
Sequence generation	Outbound email sequence generation	Generates subject lines, openers, initial email, and follow-up sequence	Implemented	`apps/web/lib/server/sequences.ts` , `packages/se` `engine/src/contracts.ts` , `apps/web/lib/server/` `provider.ts`
Sequence refinement	Partial regeneration and editing	Regenerates specific sequence parts, stores versions, and supports manual edits	Implemented	`packages/sequence-engine/src/contracts.ts` , `apps/web/lib/server/sequences.ts` , `apps/web/app/app/campaigns/[campaignId]/prosp`

Area	Feature	What it does	Status	Evidence
Quality control	Deterministic quality scoring	Stores quality reports on sequences and draft replies	Implemented	`supabase/migrations/20260328133000_quality_ch` `packages/validation/src/entities.ts`
Threads/messages	Conversation timeline	Stores outbound/inbound messages, thread state, message source, and send state	Implemented	`packages/database/migrations/0001_initial_sch` `apps/web/app/app/campaigns/[campaignId]/prosp`
Reply analysis	Intent and objection classification	Classifies replies, recommends next action, and stores rationale/confidence	Implemented	`apps/web/lib/server/replies.ts` , `packages/repl` `apps/web/lib/server/openai-reply-provider.ts`
Reply drafting	Multi-option response generation	Produces multiple response drafts, supports regeneration and manual editing	Implemented	`apps/web/lib/server/replies.ts` , `packages/repl`
Inbox connection	Gmail OAuth connection	Connects a Gmail inbox with encrypted credential storage	Implemented	`apps/web/lib/server/inbox/service.ts` , `apps/we` `provider.ts` , `supabase/migrations/20260405091500_inbox_oaut`
Inbox import	Recent thread/message import	Imports Gmail threads/messages, maps them into local threads/messages, and triggers reply analysis	Implemented	`apps/web/lib/server/inbox/service.ts` , `supabase/migrations/20260404200000_inbox_inte` `supabase/migrations/20260406131500_inbox_repl`
Inbox drafting	Push generated content into Gmail drafts	Creates Gmail drafts from generated sequence steps or reply drafts	Implemented	`apps/web/lib/server/inbox/service.ts` , `apps/we` `drafts.ts`
Sending	Real provider send	Provider-agnostic send contract exists, but sending is intentionally not implemented	Partially implemented	`packages/inbox/src/contracts.ts` , `apps/web/lib`
Billing	Plans, entitlements, checkout, portal, webhook sync	Supports Free/Pro/Agency plans, Stripe checkout/portal, subscription sync, and feature gates	Implemented	`packages/billing/src/plans.ts` , `packages/billi` `apps/web/lib/server/billing.ts` , `apps/web/app/`
Performance metrics	Campaign/workspace stats	Tracks outbound count, replies, positive replies, rates, and simple summaries	Implemented	`apps/web/lib/server/campaign-performance.ts` , `apps/web/app/app/campaigns/[campaignId]/page.`
Analytics/signals	Usage, audit, training-signal capture	Records workflow events, edits, selections, exports, and outcome signals	Implemented	`DATA_STRATEGY.md` , `packages/database/migration` `apps/web/lib/server/training-signals.ts`
Team features	Member invites/roles/removal	Supports viewing workspace members, inviting, assigning roles, and removing members	Implemented	`apps/web/app/app/settings/page.tsx` , `apps/web/` `team.ts`
Institutional controls	Retention/visibility summaries	Exposes retention preference, export/delete visibility, audit visibility, and readiness notes	Implemented	`apps/web/app/app/settings/page.tsx` , `packages/`
Data handling	Workspace export and deletion request	Exports structured workspace data and records owner-only deletion requests	Implemented	`apps/web/lib/server/data-handling.ts` , `apps/web/app/app/settings/page.tsx`
Internal tools	Debug/admin routes and demo seeding	Restricts internal operational views and allows demo data loading	Implemented	`apps/web/app/app/settings/debug/page.tsx` , `inf` `data/fixtures.ts`
Evaluation groundwork	Eval fixtures/harness	Provides regression/eval fixtures and a workflow harness	Implemented	`packages/testing/tests/run-evals.ts` , `DATA_STR`
Multi-provider routing	Second provider scaffold	Supports internal provider routing across OpenAI and Anthropic at the server layer	Implemented	`apps/web/lib/server/model-providers.ts` , `.env.`

Area	Feature	What it does	Status	Evidence
Microsoft 365 inbox	Second inbox provider	Mentioned in contracts and schema, but no provider implementation was inspected	Partially implemented	`packages/inbox/src/contracts.ts`, `supabase/migrations/20260404200000_inbox_inte`
Public sharing	Public-facing share pages	The app has shareable performance summaries in-app, but no public share pages	Not present	`apps/web/app/app/page.tsx`, `apps/web/component card.tsx`

5. How the Product Works

At a practical level, the product works like this:

1. A user signs in with Supabase-backed auth and enters a workspace-scoped application (`apps/web/lib/server/auth.ts`, `apps/web/app/sign-in/page.tsx`).
2. The user creates sender context, campaigns, and prospects in the protected app (`apps/web/app/app/onboarding/page.tsx`, `apps/web/app/app/sender-profiles/page.tsx`, `apps/web/app/app/campaigns/page.tsx`).
3. For a prospect, the app can fetch the prospect's company website, clean/extract content, summarize it into a structured company profile, and store evidence, confidence, and flags (`apps/web/lib/server/prospect-research.ts`, `packages/research-engine/src/contracts.ts`, `apps/web/lib/server/openai-research-provider.ts`).
4. The sequence engine takes sender context, campaign context, prospect company profile, tone/constraint settings, and optional performance hints to generate structured outbound email artifacts (`packages/sequence-engine/src/contracts.ts`, `apps/web/lib/server/openai-sequence-provider.ts`, `apps/web/lib/server/sequences.ts`).
5. Generated artifacts are validated, quality-scored, persisted, and shown in the prospect workflow UI, where the user can regenerate specific parts or edit content manually (`packages/validation/src/entities.ts`, `apps/web/lib/server/sequences.ts`, `apps/web/app/app/campaigns/[campaignId]/prospects/[prospectId]/page.tsx`).
6. Replies can enter the system either by manual paste/save or by Gmail import. In both paths, replies become normalized messages on a conversation thread (`apps/web/lib/server/replies.ts`, `apps/web/lib/server/inbox/service.ts`).
7. The reply engine analyzes the latest inbound message, classifies intent/objection/recommended action, and then generates multiple response draft options under explicit safety and tone constraints (`packages/reply-engine/src/contracts.ts`, `apps/web/lib/server/openai-reply-provider.ts`, `apps/web/lib/server/replies.ts`).
8. Users can edit drafts, regenerate one option, or push a selected output into Gmail as a draft. The system also tracks sent state manually or by inbox linkage, then updates campaign performance metrics and training/evaluation signals (`apps/web/lib/server/inbox/service.ts`, `apps/web/lib/server/inbox-drafts.ts`, `apps/web/lib/server/campaign-performance.ts`, `DATA_STRATEGY.md`).

Text diagram of the main flow:

```
Sign in
  -> enter workspace
  -> set up sender profile/basic mode
  -> create campaign
  -> add prospect
  -> run website research
  -> generate sequence
  -> review / edit / regenerate
  -> store outbound thread messages
  -> ingest or paste reply
  -> analyze reply
  -> generate reply drafts
  -> optionally push selected draft into Gmail
  -> track outcomes, usage, quality, and performance
```

6. User Workflow

Core outbound workflow

1. User lands on the marketing homepage and can navigate to sign-in or pricing (`apps/web/app/page.tsx` , `apps/web/app/pricing/page.tsx`).
2. User signs in with a magic-link flow and is redirected into the protected app (`apps/web/app/sign-in/page.tsx` , `apps/web/lib/server/auth.ts`).
3. If the workspace is not meaningfully set up, the user is guided through onboarding: confirm workspace, choose user type, create sender profile or select basic mode, create a first campaign, and add a first prospect (`apps/web/app/app/onboarding/page.tsx`).
4. User opens a campaign and adds or edits prospects (`apps/web/app/app/campaigns/[campaignId]/page.tsx`).
5. On a prospect detail page, the user can run research against the prospect company website (`apps/web/app/app/campaigns/[campaignId]/prospects/[prospectId]/page.tsx` , `apps/web/lib/server/prospect-research.ts`).
6. The system stores the research snapshot and uses the company profile to generate a sequence (`apps/web/lib/server/sequences.ts`).
7. The user reviews subject lines, openers, initial email, and follow-ups; can regenerate specific parts or edit text manually (`packages/sequence-engine/src/contracts.ts` , `apps/web/lib/server/sequences.ts`).
8. The user can link generated outbound content into the thread timeline and optionally create a Gmail draft rather than sending from the app (`apps/web/lib/server/inbox/service.ts` , `apps/web/lib/server/inbox-drafts.ts`).

Reply-handling workflow

1. A reply enters the system either by manual paste/save or Gmail import (`apps/web/lib/server/replies.ts` , `apps/web/lib/server/inbox/service.ts`).
2. The reply is stored as an inbound message on a conversation thread, with provider/raw/normalized data when imported from Gmail (`supabase/migrations/20260406131500_inbox_reply_ingestion.sql` , `apps/web/lib/server/inbox/service.ts`).
3. The system analyzes the latest inbound reply, storing intent, objection type, confidence, rationale, and recommended action (`packages/reply-engine/src/contracts.ts` , `apps/web/lib/server/replies.ts`).
4. The app generates three reply draft options, each governed by safety constraints such as “do not push after hard no” (`apps/web/lib/server/openai-reply-provider.ts` , `packages/reply-engine/src/contracts.ts`).
5. The user reviews the classification and drafts in the thread workspace, regenerates one option if needed, edits manually if desired, and can push a selected draft into Gmail as a draft (`apps/web/app/app/campaigns/[campaignId]/prospects/[prospectId]/page.tsx` , `apps/web/lib/server/inbox/service.ts`).

Operational/admin workflow

1. A trusted internal admin can open a restricted debug/settings route (`apps/web/app/app/settings/debug/page.tsx`).
2. The admin can inspect recent operational records, load demo seed data, and access restricted dataset export/debug functionality (`infrastructure/demo-data/fixtures.ts` , `DATA_STRATEGY.md`).

7. Technical/Operational Workflow

The product is built as a Next.js monorepo with clear internal boundaries. The practical architecture is:

- Frontend/app shell: `apps/web` contains the marketing pages, protected app pages, route handlers, and server actions (`apps/web/package.json` , `apps/web/app/...`).
- Shared domain packages: research, sequence generation, reply intelligence, billing, inbox, jobs, validation, auth, database, testing, observability, and security are separated into workspace packages (`README.md` , `ARCHITECTURE.md`).
- Validation layer: Zod contracts in `packages/validation/src/entities.ts` define the application’s shared types, schemas, and many persistence shapes. This is a strong signal that the app is designed around contract-first boundaries rather than ad hoc JSON.

- Database/storage: the schema is relational and workspace-scoped, with tables for users, workspaces, members, campaigns, prospects, research snapshots, sequences, threads/messages, reply analyses, draft replies, usage events, audit events, subscriptions, and inbox synchronization tables (`packages/database/migrations/0001_initial_schema.sql`, `supabase/migrations/...`).
- Access control: database-scoped RLS exists as a backstop, while app-layer auth and workspace checks remain primary today (`supabase/migrations/20260404121500_workspace_rls.sql`, `ARCHITECTURE.md`).
- AI/model calls: provider-specific calls are server-side only and routed through internal adapters. OpenAI is the default path; Anthropic is scaffolded as a second provider. Research, sequence generation, and reply flows all emit consistent provider metadata (`apps/web/lib/server/model-providers.ts`, `apps/web/lib/server/openai-sequence-provider.ts`, `apps/web/lib/server/openai-reply-provider.ts`, `apps/web/lib/server/openai-research-provider.ts`).
- Async/job handling: slow operations still complete in the app flow, but they persist explicit run state (`idle/running/succeeded/failed`) through a jobs contract that is designed for future queue extraction (`packages/jobs/src/index.ts`, `ARCHITECTURE.md`).
- External services: Supabase is used for auth and Postgres-oriented deployment assumptions; Stripe powers subscriptions; Gmail powers inbox connection/import and draft creation; model providers currently include OpenAI by default and Anthropic as internal routing support (`DEPLOYMENT.md`, `.env.example`, `apps/web/lib/server/billing.ts`, `apps/web/lib/server/inbox/service.ts`).
- Deployment clues: the repo is prepared for Vercel deployment, expects env-driven configuration, and uses Next.js 15 with React 19 (`apps/web/package.json`, `DEPLOYMENT.md`, `.env.example`).

---

8. Current Strengths

- The product has a coherent end-to-end use case rather than isolated demos. Research, sequence generation, reply handling, billing, and Gmail draft workflows connect inside one prospect workspace (`apps/web/app/app/campaigns/[campaignId]/prospects/[prospectId]/page.tsx`).
- The architecture is unusually disciplined for an MVP-stage repo. Shared package boundaries, schema validation, provider abstraction, and workspace-scoped data modeling are consistently visible (`ARCHITECTURE.md`, `packages/validation/src/entities.ts`, `apps/web/lib/server/model-providers.ts`).
- Safety/credibility constraints are first-class. The prompts and contracts repeatedly forbid invented claims, excessive fluff, and aggressive handling of hard-no replies (`apps/web/lib/server/openai-sequence-provider.ts`, `apps/web/lib/server/openai-reply-provider.ts`, `packages/reply-engine/src/contracts.ts`).
- The product is already instrumented for future learning. Usage, audit logs, outcome-aware signals, dataset export structures, and evaluation fixtures are implemented well beyond a bare workflow app (`DATA_STRATEGY.md`, `packages/testing/tests/run-evals.ts`).
- The app looks prepared for founder demos and early customer evaluation. Demo data, internal admin views, shareable performance summaries, onboarding, pricing, and upgrade prompts are all present in the repository (`infrastructure/demo-data/fixtures.ts`, `apps/web/app/app/settings/debug/page.tsx`, `apps/web/app/page.tsx`, `apps/web/app/pricing/page.tsx`).

---

9. Current Gaps / Limitations

- No real email sending yet. The inbox layer supports draft creation and tracking, but not provider send execution. That is explicit in the contracts and service boundary (`packages/inbox/src/contracts.ts`, `apps/web/lib/server/inbox/service.ts`).
- Microsoft 365/Outlook support is not implemented. The provider is named in contracts and schema, but the concrete implementation inspected here is Gmail-only (`packages/inbox/src/contracts.ts`, `apps/web/lib/server/inbox/gmail-provider.ts`).
- Async operations are queue-ready, not queue-backed. Run-state persistence exists, but there is no extracted worker/queue infrastructure in the repo (`packages/jobs/src/index.ts`, `ARCHITECTURE.md`).
- Some repository paths remain transitional. The architecture docs explicitly say not every repository path is Postgres-backed yet, and some lower-priority views still rely on in-memory adapters (`ARCHITECTURE.md`).
- The product is operationally serious but not yet enterprise-complete. It has team roles, institutional controls, exports, and audit visibility, but no evidence of a full compliance portal, SCIM/SSO suite, advanced governance,

or enterprise procurement features (`apps/web/app/app/settings/page.tsx` , `ARCHITECTURE.md`).

- The app has one known deployment rough edge: a documented non-blocking Next.js ESLint-plugin warning is referenced in repo work history and build notes, though it does not appear to block builds. This is an inferred operational note rather than a product feature conclusion.
- The `packages/database/migrations` folder has a numbering inconsistency with two `0006_...` files. The timestamped Supabase migrations appear to be the cleaner operational source of truth for ordering (`packages/database/migrations` , `supabase/migrations/...`).

---

10. Product Maturity Assessment

The product is best described as an **advanced MVP**.

Why:

- It has multiple complete user-facing workflows, not just mocks: onboarding, sender setup, campaigns, prospects, research, generation, reply handling, billing, and inbox connection (`apps/web/app/app/...` , `apps/web/lib/server/...`).
- It includes serious internal disciplines that many MVPs skip: audit events, usage metering, role-aware settings, evaluation harnesses, provider abstraction, structured exports, and database access-control groundwork (`DATA_STRATEGY.md` , `ARCHITECTURE.md` , `packages/validation/src/entities.ts`).
- However, it is not yet an early production product in the strictest sense because the email send path is incomplete, queueing is still preparatory, one major inbox provider is only scaffolded, and the architecture docs themselves acknowledge partial repository migration status.

---

11. Suggested “As-Is” Business Description

Outbound Copilot is a workspace-based AI outbound application for SDR teams, SaaS founders, and lead generation agencies. Today it helps teams set up sender context, research prospects from public websites, generate outbound email sequences, capture and classify inbound replies, draft follow-up responses, and track lightweight performance signals. It already includes billing, Gmail draft/import foundations, team roles, and structured data capture for future evaluation, but it is still in advanced MVP form rather than a fully complete outbound operating platform.

---

12. Appendix: Evidence from Repository

The strongest evidence for the current product state comes from:

- Product positioning and public claims: `README.md` , `apps/web/app/page.tsx` , `apps/web/app/pricing/page.tsx`
- Protected product workflows: `apps/web/app/app/page.tsx` , `apps/web/app/app/onboarding/page.tsx` , `apps/web/app/app/campaigns/[campaignId]/prospects/[prospectId]/page.tsx` , `apps/web/app/app/settings/page.tsx`
- AI workflow mechanics: `apps/web/lib/server/prospect-research.ts` , `apps/web/lib/server/sequences.ts` , `apps/web/lib/server/replies.ts`
- Provider abstraction and prompts: `apps/web/lib/server/model-providers.ts` , `apps/web/lib/server/openai-research-provider.ts` , `apps/web/lib/server/openai-sequence-provider.ts` , `apps/web/lib/server/openai-reply-provider.ts`
- Billing and pricing mechanics: `packages/billing/src/plans.ts` , `packages/billing/src/entitlements.ts` , `apps/web/lib/server/billing.ts`
- Database and access control: `packages/database/migrations/0001_initial_schema.sql` , `supabase/migrations/20260404121500_workspace_rls.sql` , `supabase/migrations/20260404200000_inbox_integration_foundation.sql` , `supabase/migrations/20260406131500_inbox_reply_ingestion.sql`
- Training/evaluation/readiness: `DATA_STRATEGY.md` , `packages/testing/tests/run-evals.ts` , `infrastructure/demo-data/fixtures.ts`

For a more detailed evidence appendix, see `generated/business-report/repo_evidence.md`.